

Lab 4: Vibe Coding — Task History

Task 1: Fix pyramid projection skew

Each cube's screen position is computed from its (row, col) grid coordinates, and the whole pyramid should come out as a symmetric triangle. In the current build the pyramid is visibly skewed from the very first frame — rows drift sideways in a way that breaks the triangular symmetry. Look at the arithmetic combining row and col in cube_center, and check whether the row offset should be using integer division or true division.

Assistant: I'll trace cube_center and the pyramid projection code, then make the smallest arithmetic fix that restores the intended symmetry.

Work recorded:

- Edited two files and ran three commands:
- `rg -n "cube_center|row\s*[/|*+-].*col|col\s*[/|*+-].*row" .`
- `git status --short; Get-Content Lab_4/game.py -TotalCount 45; Get-Content SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py -TotalCount 45`
- Wrote ./Lab_4/game.py (+1, -1).
- Wrote ./SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py (+1, -1).
- Ran `rg -n -A1 "def cube_center" Lab_4/game.py SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py; git -C SWE_Lab_PES1UG24AM218 status --short.`
- Changed `row // 2` to `row / 2` in cube_center in both copies. No tests were run.

Result: True division keeps odd-row offsets centered, restoring the pyramid's symmetry.

Task 2: Implement cube_palette(level)

Called once per frame in draw, as `colors = cube_palette(self.level)` or `DEFAULT_PALETTE`. It receives the current level number and should return a list of `TARGET + 1` (currently 3) (r, g, b) colors — one per stage from unpainted to fully painted — or `None` to keep `DEFAULT_PALETTE`. Idea: return a different 3-color palette for each level.

Assistant: I'll inspect the game's drawing and level progression so the palette changes match the existing level numbering and color conventions.

Work recorded:

- Ran two commands:
- `rg -n "cube_palette|DEFAULT_PALETTE|self\.level|TARGET|def draw" Lab_4/game.py SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py; Get-Content Lab_4/game.py`
- Wrote ./Lab_4/game.py (+7, -1).
- Wrote ./SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py (+7, -1).
- Ran `rg -n -A10 "def cube_palette" Lab_4/game.py SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py.`

- Implemented `cube_palette(level)` in both copies of `game.py`.
- It returns a three-color palette for each level, cycling through four distinct palettes; level 1 uses `DEFAULT_PALETTE`.
- No tests were run.

Task 3: Implement `on_cube_completed(cell)`

Called from `paint()` the instant a specific cube first reaches its target stage — not on every hop onto it, only the hop that finishes it. It receives the `(row, col)` cell that was just completed. Its return value is ignored. Idea: a brief flash on that cube, or a small bonus beyond the 25 points already awarded per paint step.

Assistant: I'll implement the flash as a short per-cell highlight, keyed by the `(row, col)` passed to the callback, and blend it into that cube's top color during drawing. That keeps the effect tied to the completion event without changing score or game state.

Work recorded:

- Wrote `./Lab_4/game.py (+10, -1)`.
- Wrote `./SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py (+10, -1)`.
- Ran `rg -n -A8 "def on_cube_completed|flash_until|COMPLETION_FLASH" Lab_4/game.py SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py`.
- Added a per-cell completion flash in both `game.py` copies.
- The flash lasts 300 ms, is triggered by `on_cube_completed(cell)`, and fades during drawing.
- No tests were run.

Task 4: Implement `bonus_life_threshold()`

Called every frame in `update()`. It takes no arguments and should return an integer score value, or `None` to disable bonus lives entirely. Whenever the score crosses a multiple of that value for the first time, one life is awarded automatically — the bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose the threshold. Idea: return 1000.

Assistant: I'll set the threshold to 1000 points in both game copies, matching the existing score-crossing logic.

Work recorded:

- Wrote `./Lab_4/game.py (+1, -1)`.
- Wrote `./SWE_Lab_PES1UG24AM218/Lab_4/35_qbert/game.py (+1, -1)`.
- Set `bonus_life_threshold()` to return 1000 in both `game.py` copies. No tests were run.